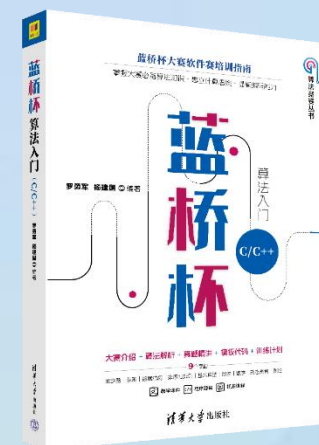


蓝桥杯大赛辅导

第一讲 算法入门

沈俊



主要内容

一、算法和算法的性能

二、前缀和

三、差分

四、贪心算法

一、算法

- 1、**算法的定义：对特定问题求解步骤的一种描述，是指令的有限序列。**
- 2、**算法的5个特征：**
 - (1)**输入：一个算法有零个或多个输入。**
 - (2)**输出：一个算法有一个或多个输出。**
 - (3)**有穷性：一个算法必须在执行有穷步之后结束，且每一步都在有穷时间内完成。**
 - (4)**确定性：算法中的每一条指令必须有确切的含义，对于相同的输入只能得到相同的输出。**
 - (5)**可行性：算法描述的操作可以通过已经实现的基本操作执行有限次来实现。**

3、算法性能

一个算法的优劣可以用**空间复杂度**与**时间复杂度**来衡量。

复杂度和大O记号

- **$O(1)$** 计算时间是一个常数，和问题的规模 n 无关。
 - 用公式计算时，一次计算的复杂度就是 $O(1)$ ，例如hash算法。
 - 在矩阵 $A[M][N]$ 中查找 i 行 j 列的元素，只需要一次访问 $A[i][j]$ 。
- **$O(\log n)$** 计算时间是对数。
 - 以2为底的对数，每一步计算后，问题的规模减小一倍。例如在一个长度为 n 的有序数列中查找某个数，用折半查找的方法，只需要 $\log n$ 次就能找到。
- **$O(n)$** 计算时间随规模 n 线性增长。
 - 在很多情况下，这是算法能达到的最优复杂度，因为对输入的 n 个数，程序一般需要处理所有的数，即计算 n 次。例如查找一个无序数列中的某个数，可能需要检查所有的数。

- $O(n \log n)$ 算法可能达到的最优复杂度。
 - 快速排序。
- $O(n^2)$ 一个两重循环的算法，复杂度是 $O(n^2)$ 。
 - 冒泡排序。
- $O(n^3)$ 、 $O(n^4)$ 等等。
- $O(2^n)$ 一般对应集合问题。
 - 例如一个集合中有 n 个数，要求输出它的所有子集。
- $O(n!)$ 在排列问题中，如果要求输出所有的排列，复杂度是 $O(n!)$

问题规模和可用算法

问题规模 n	可用算法的时间复杂度					
	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$	$O(n!)$
$n \leq 11$	✓	✓	✓	✓	✓	✓
$n \leq 25$	✓	✓	✓	✓	✓	×
$n \leq 5000$	✓	✓	✓	✓	×	×
$n \leq 10^6$	✓	✓	✓	×	×	×
$n \leq 10^7$	✓	✓	×	×	×	×
$n > 10^8$	✓	×	×	×	×	×

例题1.1：可获得的最小取值【lanqiaoOJ 3142】

- **问题描述：**有一个长度为 n 的数组 a ，进行 k 次操作来取出数组中的元素。每次操作必须选择以下两种操作之一：

(1) 取出数组中的最大元素；

(2) 取出数组中的最小元素和次小元素。

要求在进行完 k 次操作后，取出的数的和最小。

- **输入：**第一行输入两个整数 n 和 k ，表示数组长度和操作次数。第二行输入 n 个整数表示数组 a 。

- **数据范围：** $3 \leq n \leq 2 \times 10^5$ ， $1 \leq a_i \leq 10^9$ ， $1 \leq k \leq 99999$ ， $2k < n$ 。

- **输出：**输出一个整数表示最小的和。

- **样例输入：**

5 1

2 3 1 10 6

- **样例输出：**

3

解法一（贪心算法）：

```
#include<bits/stdc++.h>
using namespace std;
const int N = 200010;
long long a[N];
int main(){
    int n, k, p, q;
    cin >> n >> k;
    for (int i = 1; i <= n; i++)
        scanf("%lld",&a[i]);
    sort(a + 1, a + 1 + n);
```

```
    long long ans = 0;
    p = 1; q = n;
    for (int i = 1; i <= k; i++)
        if (a[p] + a[p+1] < a[q]){
            ans += a[p] + a[p+1];
            p += 2;
        }
        else{
            ans += a[q];
            q--;
        }
    cout << ans;
    return 0;
}
```

7 3

1 1 1 3 1 1 1

解法二（枚举算法）：

```
#include<bits/stdc++.h>
using namespace std;
const int N = 200010;
long long a[N];
int main(){
    int n, k;
    long long ans = 1e18,sum;
    cin >> n >> k;
    for (int i = 1; i <= n; i++)
        scanf("%lld",&a[i]);
    sort(a + 1, a + 1 + n);
```

```
    for (int p = 0; p <= k; p++){
        sum = 0;
        for(int i = n; i >= n-p+1; i--)
            sum += a[i];
        for(int i = 1; i <= 2*(k-p); i++)
            sum += a[i];
        ans = min(sum,ans);
    }
    cout << ans;
    return 0;
}
```

二、前缀和

- 一个长度为n的数组 $a[1] \sim a[n]$ ，前缀和 $sum[i]$ 等于 $a[1] \sim a[i]$ 的和：

$$sum[i] = a[1] + a[2] + \dots + a[i]$$

- 利用递推，可以在 $O(n)$ 时间内求得所有前缀和：

$$sum[i] = sum[i-1] + a[i]$$

- 预计算出前缀和，能利用它快速计算出数组中任意一个区间 $a[i] \sim a[j]$ 的和：

$$a[i] + a[i+1] + \dots + a[j-1] + a[j] = sum[j] - sum[i-1]$$

复杂度为 $O(n)$ 的区间求和计算，优化到了 $O(1)$ 的前缀和计算。

二、前缀和

- 在算法竞赛中很常见，蓝桥杯大赛中几乎必考。
 - (1) 原理简单，方便在很多场景下应用，与其他考点结合。
 - (2) 可以考核不同层次的能力。前缀和的题目一般也能用暴力法求解，暴力法能通过50%左右的测试，用前缀和优化后能通过70%~100%的测试。

题解

- 首先从小到大排序；然后进行两种操作：操作（1）在a[]的尾部选一个数，操作（2）在a[]的头部选2个数。
- 设操作（2）做p次，操作（1）做k-p次，求和：

$$\sum_{i=1}^{2p} a_i + \sum_{i=n+p-k+1}^n a_i$$

- 为了找最小的和，需要把所有的p都试一遍。
- **方法1：**直接按公式计算，验证一个p的计算量是O(n)的，验证所有的p， $1 \leq p \leq k$ ，总计算量O(kn)，超时。
- **方法2：**公式的两个部分就是前缀和，分别等于sum[2p]、sum[n]-sum[n+p-k]。提前算出前缀和sum[]，那么验证一个p的时间是O(1)的，验证所有p的总计算量是O(n)的。

解法三：利用前缀和进行DP

```
#include<bits/stdc++.h>
using namespace std;
const int N = 200010;
long long a[N],sum[N];          //sum[]是a[]的前缀和
int main(){
    int n, k;
    cin >> n >> k;
    for (int i = 1; i <= n; i++) scanf("%lld",&a[i]);
    sort(a + 1, a + 1 + n);
    for (int i = 1; i <= n; i++) sum[i] = sum[i-1] + a[i];
    long long ans = 1e18;
    for (int p = 1; p <= k; p++)
        ans = min(sum[n] - sum[n+p-k] + sum[2*p], ans);
    cout << ans;
    return 0;
}
```


例题1.2：重新排序【lanqiaoOJ 2128】

- 问题描述：给定一个数组A和一些查询 L_i, R_i ，求数组中第 L_i 至第 R_i 个元素之和。小蓝觉得这个问题很无聊，于是他想重新排列一下数组，使得最终每个查询结果的和尽可能地大。小蓝想知道相比原数组，所有查询结果的总和最多可以增加多少？
- 输入：输入第一行包含一个整数n。第二行包含n个整数 $A_1, A_2, \dots, A_n$ ，相邻两个整数之间用一个空格分隔。第三行包含一个整数m表示查询的数目。接下来m行，每行包含两个整数 L_i, R_i ，相邻两个整数之间用一个空格分隔。
- 输出：输出一行一个整数表示答案。

• 输出样例

输出样例

5

4

1 2 3 4 5

2

1 3

2 5

题解

- 对于30%的评测用例, $n, m \leq 50$;
 - 对于50%的评测用例, $n, m \leq 500$;
 - 对于70%的评测用例, $n, m \leq 5000$;
 - 对于所有评测用例, $1 \leq n, m \leq 10^5$, $1 \leq A_i \leq 10^6$, $1 \leq L_i \leq R_i \leq n$.
-
- m 个查询可以统一处理, 读入 m 个查询后, 每个 $a[i]$ 被查询了多少次就知道了。
用 $cnt[i]$ 记录 $a[i]$ 被查询的次数, $cnt[i] * a[i]$ 就是 $a[i]$ 对总和的贡献。
 - (1) 通过70%的测试。 (对于70%的评测用例, $n, m \leq 5000$)
 - (2) 通过100%的测试。 (对于所有评测用例, $1 \leq n, m \leq 10^5$, $1 \leq A_i \leq 10^6$, $1 \leq L_i \leq R_i \leq n$)

解法一：枚举所有排列，求最大差值

解法二：

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define e 100005
ll n, a[e] = { 0 }, cf[e] = { 0 }, m, ysum=0, gsum=0;
bool cmp(ll x, ll y) { return x > y; }
int main() {
    cin >> n;
    for (ll i = 1; i <= n; i++) cin >> a[i];
    cin >> m;
    ll l, r;
    for (ll i = 1; i <= m; i++) {
        cin >> l >> r;
        for (int j = l; j <= r; j++)
            cf[j]++;
    }
```

```
for (ll i = 1; i <= n; i++) {
    ysum = ysum + a[i] * cf[i];
}
sort(a + 1, a + n + 1, cmp);
sort(cf + 1, cf + n + 1, cmp);
for (ll i = 1; i <= n; i++) {
    gsum = gsum + a[i] * cf[i];
}
cout << gsum - ysum;
return 0;
}
```


三、差分

- 与一维数组a[]对应的差分数组d[]的定义：

$$d[k] = a[k] - a[k-1]$$

即差分数组d[]是原数组a[]的相邻元素的差。

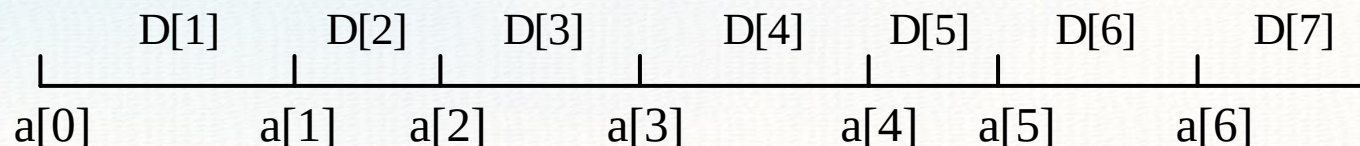
- 根据d[]的定义，可以反过来推出：

$$a[k] = d[1] + d[2] + \dots + d[k]$$

即a[]是d[]的前缀和， **“差分是前缀和的逆运算”**。

前缀和与差分

- ✓ 一维差分数组 $D[k] = a[k] - a[k-1]$, 即原数组 $a[]$ 的相邻元素的差



$$a[k] = D[1] + D[2] + \dots + D[k]$$

$a[]$ 是 $D[]$ 的前缀和

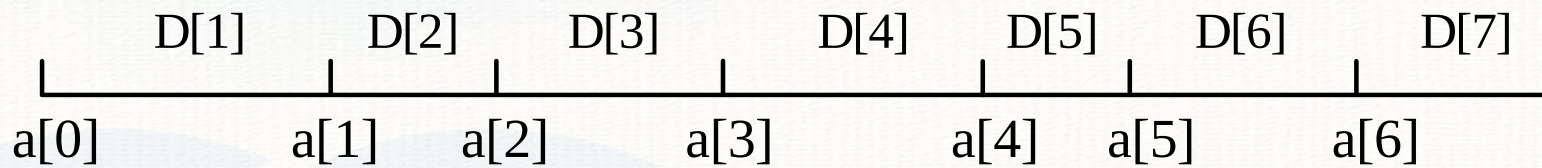
- ✓ 差分是前缀和的逆运算: 把求 $a[k]$ 转化为求 D 的前缀和

差分数组：提升修改的效率

✓ 把区间 $[L, R]$ 内每个元素 $a[i]$ 加上 d ，只需把对应的 $D[i]$ 做以下操作：

(1) 把 $D[L]$ 加上 d : $D[L] += d$

(2) 把 $D[R+1]$ 减去 d : $D[R+1] -= d$



✓ 利用 $D[]$ ，能极快解决修改区间 $[L, R]$ 内元素的目的。原来需要 $O(n)$ 次计算，现在只需要 $O(1)$ 。

✓ 说明：前缀和 $a[x] = D[1] + D[2] + \dots + D[x]$ ，有：

(1) $1 \leq x < L$ ，前缀和 $a[x]$ 不变；

(2) $L \leq x \leq R$ ，前缀和 $a[x]$ 增加了 d ；

(3) $R < x \leq N$ ，前缀和 $a[x]$ 不变，因为被 $D[R+1]$ 中减去的 d 抵消了。

解法三:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define e 100005
ll n, a[e] = { 0 }, cf[e] = { 0 }, m,
ll ysum=0,gsum=0;
bool cmp(ll x, ll y) { return x > y; }
int main() {
    cin >> n;
    for (ll i = 1; i <= n; i++) cin >> a[i];
    cin >> m;
    ll l, r;
    for (ll i = 1; i <= m; i++) {
        cin >> l >> r;
        cf[l]++;    cf[r + 1]--;
    }
```

```
    for (ll i = 1; i <= n; i++) {
        cf[i] = cf[i] + cf[i - 1];
    }
    for (ll i = 1; i <= n; i++) {
        ysum = ysum + a[i] * cf[i];
    }
    sort(a + 1, a + n + 1, cmp);
    sort(cf + 1, cf + n + 1, cmp);
    for (ll i = 1; i <= n; i++) {
        gsum = gsum + a[i] * cf[i];
    }
    cout << gsum - ysum;
}
```


四、贪心算法

1、贪心算法的策略

- 贪心策略不从整体最优考虑，而总是某种意义上是局部最优的方面作出选择。
- 即：贪心策略总是作出在当前看来最好的选择。
- 目标：得到的最终结果也是整体最优的。
- 局限性：贪心算法未必能对所有问题都得到整体最优解
- 贪心算法对许多问题能产生整体最优解，且效率较高

四、贪心算法

2、贪心算法的前提和要素

- 问题的整体最优解总可以通过一系列局部最优的选择达到。
- 贪心算法的前提：
 - (1) **第一步的局部选择**：选择当前的局部最优，保证该局部最优包含在整体最优的一系列策略之中。
 - (2) **下一步的继续选择**：整体最优的下一步还可以继续通过局部最优达到。
- 两个要素：
 - (1) 贪心选择性质
 - (2) 最优子结构性质

例1.3: P2240 【深基12.例1】部分背包问题

题目描述

 复制Markdown  展开

阿里巴巴走进了装满宝藏的藏宝洞。藏宝洞里面有 N ($N \leq 100$) 堆金币, 第 i 堆金币的总重量和总价值分别是 m_i, v_i ($1 \leq m_i, v_i \leq 100$)。阿里巴巴有一个承重量为 T ($T \leq 1000$) 的背包, 但并不一定有办法将全部的金币都装进去。他想装走尽可能多价值的金币。所有金币都可以随意分割, 分割完的金币重量价值比 (也就是单位价格) 不变。请问阿里巴巴最多可以拿走多少价值的金币?

输入格式

第一行两个整数 N, T 。

接下来 N 行, 每行两个整数 m_i, v_i 。

输出格式

一个实数表示答案, 输出两位小数

输入输出样例

输入 #1

 复制

```
4 50
10 60
20 100
30 120
15 45
```

输出 #1

 复制

```
240.00
```

例1.3: P2240 【深基12.例1】 部分背包问题

```
#include<bits/stdc++.h>

using namespace std;

int n;//有n堆金币

double t,v[101],w[101],a[101],ans;

int main() {
    cin>>n>>t;
    for(int i=1;i<=n;i++)    {
        cin>>w[i]>>v[i];      a[i]=v[i]/w[i];//求性价比
    }
    for(int i=1;i<=n;i++)//性价比排序
        for(int j=1;j<n;j++)
            if(a[j]<a[j+1]){
                swap(a[j],a[j+1]); swap(v[j],v[j+1]); swap(w[j],w[j+1]);
            }
}
```


例1.3: P2240 【深基12.例1】 部分背包问题

```
for(int i=1;i<=n;i++)
    if(t-w[i]>-0.000001){
        t-=w[i];
        ans+=v[i];
    }
    else{
        ans+=t*a[i];//把金币分割
        break;
    }
printf("%.2lf",ans);
return 0;
}
```

例1.4： 2、P1223 排队接水

题目描述

 复制Markdown  展开

有 n 个人在一个水龙头前排队接水，假如每个人接水的时间为 T_i ，请编程找出这 n 个人排队的一种顺序，使得 n 个人的平均等待时间最小。

输入格式

第一行为一个整数 n 。

第二行 n 个整数，第 i 个整数 T_i 表示第 i 个人的接水时间 T_i 。

输出格式

输出文件有两行，第一行为一种平均时间最短的排队顺序；第二行为这种排列方案下的平均等待时间（输出结果精确到小数点后两位）。

输入输出样例

输入 #1

 复制

```
10
56 12 1 99 1000 234 33 55 99 812
```

输出 #1

 复制

```
3 2 7 8 1 4 9 6 10 5
291.90
```


例1.4: P1223 排队接水

```
#include <iostream>
#include <cstdio>
#include <algorithm>
using namespace std;
double sum;
long int n;
long long int t[1001];
double ans;
int main(int argc, const char * argv()){
    int x;
    cin>>n;
```

```
    for(int i=1;i<=n;i++) {
        cin>>x;
        t[i]=x*1001+i;
    }
    sort(t+1,t+1+n);
    for(int j=1;j<=n;j++) {
        cout<<t[j]%1001<<" ";
        sum+=t[j]/1001*(n-j);
    }
    cout<<endl;
    ans=sum/n;
    printf(" %0.2lf",ans);
    return 0;
}
```

例1.5: P1803 凌乱的yyy

题目背景

 复制Markdown  展开

快 noip 了, yyy 很紧张!

题目描述

现在各大 oj 上有 n 个比赛, 每个比赛的开始、结束的时间点是知道的。

yyy 认为, 参加越多的比赛, noip 就能考的越好 (假的)。

所以, 他想知道他最多能参加几个比赛。

由于 yyy 是蒟蒻, 如果要参加一个比赛必须善始善终, 而且不能同时参加 2 个及以上的比赛。

输入格式

第一行是一个整数 n , 接下来 n 行每行是 2 个整数 a_i, b_i ($a_i < b_i$), 表示比赛开始、结束的时间。

输出格式

一个整数最多参加的比赛数目。

输入输出样例

输入 #1

 复制

```
3
0 2
2 4
1 3
```

输出 #1

 复制

```
2
```


例1.5: P1803 凌乱的yyy

```
#include <cstdio>
#include <algorithm>
using namespace std;
```

```
struct Match{
    int start;
    int end;
}M[1000005];
```

```
bool compare(Match a, Match b){
    return a.end < b.end;
}
```

```
int main(){
```

```
    int n,i,j, endT = 0, ans=0;
    scanf("%d",&n);
    for (i=0;i<n;i++)
        scanf("%d %d",&M[i].start,&M[i].end);
    sort(M,M+n,compare);
    for(i = 0;i < n;i++)
        if(endT <= M[i].start) {
            endT = M[i].end;
            ans++;
        }
```

```
    printf("%d\n",ans);
    return 0;
```

```
}
```

例1.6： [NOIP2004 提高组] 合并果子

题目描述

 复制Markdown  展开

在一个果园里，多多已经把所有的果子打了下来，而且按果子的不同种类分成了不同的堆。多多决定把所有的果子合成一堆。

每一次合并，多多可以把两堆果子合并到一起，消耗的体力等于两堆果子的重量之和。可以看出，所有的果子经过 $n - 1$ 次合并之后，就只剩下一堆了。多多在合并果子时总共消耗的体力等于每次合并所耗体力之和。

因为还要花大力气把这些果子搬回家，所以多多在合并果子时要尽可能地节省体力。假定每个果子重量都为 1，并且已知果子的种类数和每种果子的数目，你的任务是设计出合并的次序方案，使多多耗费的体力最少，并输出这个最小的体力耗费值。

例如有 3 种果子，数目依次为 1，2，9。可以先将 1、2 堆合并，新堆数目为 3，耗费体力为 3。接着，将新堆与原先的第三堆合并，又得到新的堆，数目为 12，耗费体力为 12。所以多多总共耗费体力 $= 3 + 12 = 15$ 。可以证明 15 为最小的体力耗费值。

输入格式

共两行。
第一行是一个整数 $n(1 \leq n \leq 10000)$ ，表示果子的种类数。
第二行包含 n 个整数，用空格分隔，第 i 个整数 $a_i(1 \leq a_i \leq 20000)$ 是第 i 种果子的数目。

输出格式

一个整数，也就是最小的体力耗费值。输入数据保证这个值小于 2^{31} 。

输入输出样例

输入 #1

 复制

输出 #1

```
3
1 2 9
```

```
15
```

说明/提示

对于 30% 的数据，保证有 $n \leq 1000$ ；
对于 50% 的数据，保证有 $n \leq 5000$ ；
对于全部的数据，保证有 $n \leq 10000$ 。

例1.6: [NOIP2004 提高组] 合并果子

```
template<typename item>
```

```
class smallest_heap{
```

```
    private:
```

```
        item heap[10001];
```

```
        int len;
```

```
    public:
```

```
        smallest_heap();
```

```
        void push(item const &);
```

```
        void pop();
```

```
        item top();
```

```
        int size();
```

```
        bool empty();
```

```
};
```

```
template<typename item>
```

```
smallest_heap<item>::smallest_heap(){
```

```
    len=0;
```

```
    memset(heap,0,sizeof(heap));
```

```
}
```

```
template<typename item>
```

```
void smallest_heap<item>::push(item const &n){
```

```
    heap[++len]=n;
```

```
    int son=len,father=son/2;
```

```
    while(heap[son]<heap[father] &&  
father>=1){
```

```
        swap(heap[son],heap[father]);
```

```
        son=father,father=son/2;
```

```
    }
```

```
}
```

例1.6: [NOIP2004 提高组] 合并果子

```
template<typename item>
```

```
void smallest_heap<item>::pop(){
```

```
    swap(heap[1],heap[len]);
```

```
    heap[len--]=0;
```

```
    int father=1,son=2;
```

```
    while(son<=len){
```

```
        if(son<len && heap[son]>heap[son+1])
```

```
            son++;
```

```
        if(heap[father]>heap[son]){
```

```
            swap(heap[father],heap[son]);
```

```
            father=son,son=father*2;
```

```
        }else break;
```

```
    }
```

```
}
```


例1.6: [NOIP2004 提高组] 合并果子

```
template<typename item>
```

```
item smallest_heap<item>::top(){
```

```
    return heap[1];
```

```
}
```

```
template<typename item>
```

```
int smallest_heap<item>::size(){
```

```
    return len;
```

```
}
```

```
template<typename item>
```

```
bool smallest_heap<item>::empty(){
```

```
    return len;
```

```
}
```

```
smallest_heap<int> h;
```

```
int n,ans,a,x,y;
```

```
int main(){
```

```
    cin>>n;
```

```
    for(int i=1;i<=n;i++){
```

```
        cin>>a;        h.push(a);
```

```
    }
```

```
    while(h.size()>1){
```

```
        x=h.top(); h.pop();
```

```
        y=h.top(); h.pop();
```

```
        h.push(x+y); ans+=x+y;
```

```
    }
```

```
    cout<<ans;
```

```
    return 0;
```

```
}
```


大根堆:

```
template<typename item>
```

```
class largest_heap{
```

```
    private:
```

```
        item heap[10001];
```

```
        int len;
```

```
    public:
```

```
        largest_heap();
```

```
        void push(item const &);
```

```
        void pop();
```

```
        item top();
```

```
        int size();
```

```
        bool empty();
```

```
};
```

```
template<typename item>
```

```
largest_heap<item>::largest_heap(){
```

```
    len=0;
```

```
    memset(heap,0,sizeof(heap));
```

```
}
```

```
template<typename item>
```

```
void largest_heap<item>::push(item const &n){
```

```
    heap[++len]=n;
```

```
    int son=len,father=son/2;
```

```
    while(heap[son]>heap[father] && father>=1){
```

```
        swap(heap[son],heap[father]);
```

```
        son=father,father=son/2;
```

```
    }
```

```
}
```


大根堆:

```
template<typename item>
void largest_heap<item>::pop(){
    swap(heap[1],heap[len]);
    heap[len--]=0;
    int father=1,son=2;
    while(son<=len){
        if(son<len && heap[son]<heap[son+1])
            son++;
        if(heap[father]<heap[son]){
            swap(heap[father],heap[son]);
            father=son,son=father*2;
        }else break;
    }
}
```

```
template<typename item>
item largest_heap<item>::top(){
    return heap[1];
}

template<typename item>
int largest_heap<item>::size(){
    return len;
}

template<typename item>
bool largest_heap<item>::empty(){
    return len;
}
```

例1.7: P3817 小A的糖果

题目描述

 复制Markdown  展开

小 A 有 n 个糖果盒，第 i 个盒中有 a_i 颗糖果。

小 A 每次可以从其中一盒糖果中吃掉一颗，他想知道，要让任意两个相邻的盒子中糖的个数之和都不大于 x ，至少得吃掉几颗糖。

输入格式

输入的第一行是两个用空格隔开的整数，代表糖果盒的个数 n 和给定的参数 x 。

第二行有 n 个用空格隔开的整数，第 i 个整数代表第 i 盒糖的糖果个数 a_i 。

输出格式

输出一行一个整数，代表最少要吃掉的糖果的数量。

输入输出样例

输入 #1

 复制

```
3 3
2 2 2
```

输出 #1

 复制

```
1
```

输入 #2

 复制

```
6 1
1 6 1 2 0 4
```

输出 #2

 复制

```
11
```


例1.7: P3817 小A的糖果

输入 #3

复制

```
5 9
3 1 4 1 5
```

输出 #3

复制

```
0
```

说明/提示

样例输入输出 1 解释

吃掉第 2 盒中的一个糖果即可。

样例输入输出 2 解释

第 2 盒糖吃掉 6 颗，第 4 盒吃掉 2 颗，第 6 盒吃掉 3 颗。

数据规模与约定

- 对于 30% 的数据，保证 $n \leq 20$, $a_i, x \leq 100$ 。
- 对于 70% 的数据，保证 $n \leq 10^3$, $a_i, x \leq 10^5$ 。
- 对于 100% 的数据，保证 $2 \leq n \leq 10^5$, $0 \leq a_i, x \leq 10^9$ 。

例1.7: P3817 小A的糖果

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    long long n,x,a[1000001],ans = 0;
    cin >> n >> x;
    cin >> a[1];
    if(a[1] > x) {
        ans += a[1]-x;    a[1] = x;
    }
    for(int i=2;i<=n;i++)    {
        cin>>a[i];
        if(a[i]+a[i-1]>x){
            ans += a[i]+a[i-1]-x;    a[i] = x-a[i-1];
        }
    }
    cout << ans << endl;
    return 0;
}
```